

P1154R2

Type traits for structural comparison

Published Proposal, 2019-03-10

Authors:

[Arthur O'Dwyer](#)

[Jeff Snyder](#)

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Draft Revision:

7

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d1154-comparable-traits.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d1154-comparable-traits.html

Abstract

Now that [P0732R2 "Class Types in Non-Type Template Parameters"](#) has been adopted, we propose the new type-trait `has_strong_structural_equality<T>`. Code is more robust and maintainable when we can `static_assert` this property.

Table of Contents

- 1 Changelog**
- 2 Strong structural equality should be static-assertable**
- 3 This feature benefits from compiler support**
- 4 Provide just the one type trait**
- 5 Proposed wording for C++2a**
- 6 Straw polls taken of LEWG in Kona (2019-02-18)**

References

Informative References

§ 1. Changelog

- R2 (post-Kona):
 - Removed five of the six proposed type-traits at LEWG's request (see [§6 Straw polls taken of LEWG in Kona \(2019-02-18\)](#)). LEWG voted to forward this version to LWG for C++2a.

§ 2. Strong structural equality should be static-assertable

The concept of "having strong structural equality" that was introduced by [\[P0732\]](#) will become important to programmers. Take this class type for example:

```
template<std::size_t N>
struct fixed_string
{
    constexpr fixed_string(const char (&s)[N+1])
        { std::copy_n(s, N+1, m_data); }
    auto operator<=>(const fixed_string &) const = default;
    char m_data[N+1];
};
```

This type's `operator<=>` is a "structural comparison operator" because it is defaulted, and invokes only other structural comparison operators. Since it yields `strong_ordering`, it also has what we might call "strong structural ordering", although this term is not introduced by P0732. "Strong structural ordering" is a stricter requirement than P0732's "strong structural equality," which is the prerequisite to use a user-defined type as a non-type template parameter.

C++ should permit the programmer to test the presence or absence of this property. Example:

```
static_assert(std::has_strong_structural_equality_v< fixed_string<5> >);
```

This permits maintainability-minded programmers to express their intention in code.

```
template<std::size_t N>
struct broken_fixed_string
{
    constexpr broken_fixed_string(const char (&s)[N+1])
        { std::copy_n(s, N+1, m_data); }
    auto operator<=>(const broken_fixed_string &rhs) const
        { return std::memcmp(m_data, rhs.m_data, N+1) <=> 0; }
    char m_data[N+1];
};

static_assert(std::has_strong_structural_equality_v< broken_fixed_string<5>
    "broken_fixed_string lacks the strong structural equality we expected")

// ... possibly many lines of code here ...
// ... possibly written by a different programmer ...

template<auto V> struct A {};
A<broken_fixed_string("hello")> a;
```

In the snippet above, we get a nice descriptive `static_assert` failure, instead of an unfriendly spew of diagnostics on the line that tries to instantiate `A`.

§ 3. This feature benefits from compiler support

P1154R0 claimed that this type-trait could not be implemented without a compiler builtin. In fact, `has_strong_structural_equality` can be implemented according to standard C++17:

```
template<auto> struct A {};  
  
template<class T, template<T> class = A> using B = void;  
  
template<class T, class = void>  
struct HasStrongStructuralEquality : std::false_type {};  
  
template<class T>  
struct HasStrongStructuralEquality<T, B<T>> : std::true_type {};  
  
static_assert(HasStrongStructuralEquality<int >::value);  
static_assert(!HasStrongStructuralEquality<std::string >::value);
```

This code relies on subtle and maybe-uncertain rules governing when `template<auto> struct A` is a valid argument for a parameter of type `template<T> class`.

- GCC currently does not support this code — that is, they fail the second `static_assert`.
- MSVC flatly rejects it because they don't support `auto` template parameters yet.
- Clang accepts this code. The worst that can be said of Clang is that they give the wrong answer for `HasStrongStructuralEquality<int&&>`, but that can easily be worked around on the library side.

Right now the burden is on the application programmer to know this trivia and come up with workarounds for GCC and MSVC. We propose to simplify the programmer's job by putting this trait into the standard library, where the burden will be on the library to get it right (probably using a compiler builtin).

§ 4. Provide just the one type trait

P1154R1 proposed six type-traits, with implementations shown in terms of a hypothetical compiler builtin `__has_structural_comparison(T)`. After the adoption of [P1185], the general notion of "structural comparison" is no longer meaningful; the new core-language feature is "strong structural equality" (only).

Strong structural equality may be provided by a structural (defaulted) `operator<=>` or by a structural (defaulted) `operator==`. In the latter case, the type might not support `operator<=>` at all.

Therefore we propose only the following type-trait, with accompanying `_v` variable template. For exposition purposes only, we provide a sample implementation in terms of a hypothetical compiler builtin `__has_strong_structural_equality(T)`.

```
template<class T> struct has_strong_structural_equality :  
    bool_constant< __has_strong_structural_equality(T) > {};
```

§ 5. Proposed wording for C++2a

Add one new entry to Table 47 in [meta.unary.prop]:

Template	Condition	Preconditions
<code>template<class T> struct has_strong_structural_equality;</code>	The type <code>T</code> has strong structural equality (10.10.1).	<code>T</code> shall be a complete type, cv <code>void</code> , or an array of unknown bound.

§ 6. Straw polls taken of LEWG in Kona (2019-02-18)

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>We want all of the traits [from P1154R1] (rather than just strong structural equality).</i>	0	2	4	0	3
<i>Drop everything but <code>has_strong_structural_equality[_v]</code>, make sure to coordinate with [P1185], and forward to LWG for C++20.</i>	3	5	1	0	0

§ References

§ Informative References

[P0732]

Jeff Snyder; Louis Dionne. [Class Types in Non-Type Template Parameters](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0732r2.pdf). June 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0732r2.pdf>

[P1185]

Barry Revzin. [<=> != ==](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1185r1.html). January 2019. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1185r1.html>